Branching and Release Management Standard

Document ID	ENG-006
Department	Engineering
Document type	Standard
Owner	Director of Platform Engineering
Version	3.3
Status	APPROVED
Effective / last reviewed	2026-03-12
Audience	Engineering staff
Related documents	ENG-001, ENG-003, ENG-007, CS-006

Summary. Standard for source branching, versioning, tagging and release management at Northstar Systems. Covers trunk-based development, branch naming, protection rules, release and hotfix branches, semantic versioning, signed tags, the weekly release train, freezes and product version support.

SYNTHETIC DOCUMENT - Northstar Systems is a fictional company. This document exists for development, testing and demonstration only and is not real legal, employment, security or compliance advice.

1. Purpose and Scope

This standard defines how Northstar Systems Private Limited engineering teams organise source branches, version software, tag releases and move changes into production through the weekly release train. It applies to every repository that contains code, infrastructure definitions or configuration for the Northstar Nexus Platform, Nexus Connect, Nexus Insight and internal services, and to every engineer, reviewer, release manager and engineering manager who touches them.

The objectives are to keep the main line always releasable, make releases small and boring, keep traceability from every production artefact back to a reviewed commit, and make hotfixes fast without bypassing control. Code review rules are in ENG-001, deployment mechanics and rollback in ENG-003, pipeline configuration in ENG-007 and customer-facing release communication in CS-006. Where this standard and the deployment runbook ENG-003 appear to differ on a production deployment rule, the more restrictive rule applies until the Director of Platform Engineering clarifies.

2. Definitions

- **Trunk**: the `main` branch, the single integration branch from which releases are cut.
- **Feature branch**: a short-lived branch created from `main` for one change and merged back through a pull request (PR).
- **Release branch**: a branch named `release/<version>` cut from `main` to stabilise and support a specific version.
- **Hotfix branch**: a branch named `hotfix/<ticket>` created to fix a production defect, where `<ticket>` is the NorthDesk ticket number.
- **Semantic versioning**: version numbers of the form MAJOR.MINOR.PATCH.

- **Release train**: the fixed weekly schedule on which approved changes are promoted to production, every Tuesday.
- **Release manager**: the engineer assigned each week by SRE to run the release train.
- **Feature flag**: a runtime switch that lets code be merged and deployed while remaining inactive.
- **Signed tag**: an annotated Git tag cryptographically signed by an authorised release signer.
- **LTS**: Long-Term Support version of a product. LTS versions receive 24 months of support and non-LTS versions 12 months.

3. Roles and Responsibilities

Role	Responsibilities
Developer	Creates short-lived branches, keeps PRs small, writes tests, uses feature flags for unfinished work
Reviewer	Reviews per ENG-001 and refuses merges that break this standard
Release manager	Runs the Tuesday release train, creates tags, verifies pipeline gates, coordinates rollout
Director of Platform Engineering	Owns this standard, approves exceptions to branch protection and versioning
Release signer	Small named group authorised to create signed tags, listed in NorthShip
Engineering manager	Ensures their team follows the standard and approves hotfix branches
SRE	Maintains freeze calendar, change records and rollback readiness

4. Branching Model

4.1 Trunk-based development

Northstar uses trunk-based development with short-lived feature branches. Developers create a feature branch from the latest `main`, make small changes and merge back through a reviewed PR. The recommended maximum lifetime of a feature branch is 3 days. A branch that lives longer than 3 days should be broken into smaller changes or protected by a feature flag so that the work can be merged earlier. Long-lived feature branches are not allowed for new work, because they hide integration risk and cause painful merges.

4.2 Branch naming

Branch type	Pattern	Example
Main	`main`	`main`
Feature	`feature/<ticket>-<short-description>`	`feature/NX-4412-retry-backoff`
Bug fix	`fix/<ticket>-<short-description>`	`fix/NX-4519-null-webhook-header`
Release	`release/<version>`	`release/5.1.0`
Hotfix	`hotfix/<ticket>`	`hotfix/NDESK-88123`

Branch names must be lowercase except ticket prefixes, must not contain spaces and must reference a NorthDesk or product ticket where one exists. Personal branches such as `username/experiment` may be used for experiments but must not be merged to `main` without being renamed and linked to a ticket.

4.3 Protection of main

The `main` branch is protected. Direct pushes, force pushes and branch deletion are blocked for everyone including administrators. A change reaches `main` only through a PR that meets the following conditions:

1 The author cannot approve their own change.
2 Tier-1 services require 2 approving reviewers and other services require 1, as defined in ENG-001.
3 All required NorthShip pipeline checks pass (build, unit tests, linting, dependency scan and secret scan, see ENG-007).
4 New code coverage is at least 80%.
5 Security-sensitive changes (authentication, cryptography, personal data handling) are reviewed by a Security Champion.
6 The PR description links a ticket and states the rollback approach.
7 The branch is up to date with `main` or can be merged cleanly.

PRs should have at most 400 changed lines, and PRs inactive for more than 14 days are flagged as stale by the tooling. The merge method for feature branches is squash merge, producing one commit per PR with the ticket number in the message.

4.4 Feature flags

Incomplete features must be merged behind a feature flag that defaults to off. Every flag needs an owner, a creation date, an intended removal date and a ticket for removal. Flags that remain after 90 days from full rollout should be removed. A flag that controls a security or data-handling behaviour needs a Security Champion review. A flag change in production is a Standard change if it only enables a feature for internal tenants, and a Normal change if it enables it for customers.

5. Versioning

5.1 Semantic versioning

All products, libraries and public APIs use semantic versioning MAJOR.MINOR.PATCH.

- MAJOR increases when there are incompatible changes. Public API breaking changes may only be introduced in a new major version, as required by ENG-002.
- MINOR increases for backwards-compatible features.
- PATCH increases for backwards-compatible fixes.

Pre-release versions use suffixes such as `-rc.1`. Build metadata is added by the pipeline and is never used to compare versions.

5.2 Product versions

The product line currently supports the following versions of the Nexus Platform.

Version	Type	Released	Support
4.2	LTS	2025-01-20	until 2027-01-20
4.3	non-LTS	2025-06-03	ended 2026-06-03
5.0	LTS	2026-02-10	until 2028-02-10
5.1	non-LTS	2026-08-18	until 2027-08-18

LTS versions are supported for 24 months and non-LTS versions for 12 months. The supported upgrade paths are 4.2 directly to 5.0, and 4.3 directly to 5.1; a customer on 4.3 who wants 5.0 must go via 5.1 or follow the documented path through 5.0 as described by Customer Success. Release managers must keep the upgrade matrix tested in the pipeline for every supported path.

5.3 Deprecation

Public API deprecations require at least 12 months' notice and a Sunset header, as required by ENG-002. The release notes of the MINOR version that introduces the deprecation must describe the replacement and the removal version.

6. Release Branches and Tags

6.1 When to cut a release branch

Services deployed continuously from `main` through the weekly release train do not need a release branch. A release branch `release/<version>` must be cut for any product version that is shipped to customers as a versioned package or must be supported in parallel with newer versions, which means the Nexus Platform LTS lines and non-LTS lines listed above. The release branch is cut from `main` at the agreed code-complete date, typically 2 weeks before the planned release.

6.2 Rules on a release branch

- Only bug fixes and low-risk changes approved by the release manager are accepted.
- Fixes land on `main` first and are then cherry-picked to the release branch. A fix made only on a release branch must be forward-ported to `main` within 2 business days.
- All release branch changes need the same review and pipeline gates as `main`.
- The release branch is closed when the version leaves support, and the final tag is retained.

6.3 Tagging

Every production release must be tagged on the exact commit that was built. Tags are annotated and signed by a release signer, in the form `v<MAJOR>.<MINOR>.<PATCH>`. Tags are immutable; a bad tag must be superseded by a new version and must never be deleted or moved. The pipeline refuses to promote artefacts that do not trace to a signed tag, and it attaches the SBOM and the build provenance to the release record.

Example: The 5.1 line has a bug fix. The developer merges the fix to `main` through a reviewed PR, then the release manager cherry-picks it to `release/5.1.0`, bumps the version to 5.1.1 and a release signer creates the signed tag `v5.1.1`. The pipeline builds the tag, generates the SBOM and promotes the artefact to staging.

7. The Weekly Release Train

7.1 Schedule

The release train departs weekly on Tuesday. Production deployment is allowed only in the window Tuesday to Thursday, 10:00 to 16:00 IST. No production deployments are made on Friday or on Monday morning, except for emergency changes. The following table summarises the weekly cycle.

Day and time (IST)	Activity
Monday 12:00	Change requests for Normal changes due

Day and time (IST)	Activity
Monday 15:00	Change Advisory Board (CAB) meeting
Monday afternoon	Release manager confirms the release candidate and the scope list
Tuesday 10:00 to 16:00	Release train, deployment to production through staged rollout
Wednesday and Thursday 10:00 to 16:00	Fix-forward and catch-up deployments inside the window
Friday	No production deployment (emergencies only)

7.2 Change types

- **Standard change**: pre-approved, low risk, repeatable (for example a configuration flag in an approved list). No CAB discussion needed, but a NorthDesk change ticket is created.
- **Normal change**: needs CAB approval. Requests must be submitted by Monday 12:00 IST for the Monday 15:00 IST CAB meeting.
- **Emergency change**: approved by the Engineering Director on call, with a retrospective within 1 business day.

Every production change needs a NorthDesk change ticket and a rollback plan. The release manager must verify that the ticket number is in the release record before the first rollout stage.

7.3 Staged rollout

Rollout proceeds through canary 10% for 30 minutes, then 50% for 30 minutes, then 100%. The release manager watches the NorthWatch golden signals dashboard at each stage. Automatic rollback triggers when the error rate is above 2% or p95 latency exceeds 1.5x baseline for 5 consecutive minutes. A rollback must complete within 15 minutes of the decision. After a rollback the release manager must open a NorthDesk problem ticket and update the release record with the reason, and the fix must pass through the usual PR route before the next attempt.

7.4 Freezes

No production deployments are made during the last 3 business days of each quarter or during a company-declared freeze such as a customer launch event. SRE publishes the freeze calendar at the start of each quarter. Emergency changes are still possible during a freeze but require Engineering Director on call approval and notification to the VP Engineering.

Example: The quarter ending 30 September 2026 ends on a Wednesday. The last 3 business days are Monday 28, Tuesday 29 and Wednesday 30 September. The Tuesday 29 September release train is therefore cancelled, and the last train of the quarter departs on Tuesday 22 September.

7.5 Release notes

The release manager prepares release notes for each train, listing user-visible changes, bug fixes, deprecations and known issues, with ticket links. Customer Success reviews notes before they are shared with customers, as in CS-006. Notes must not contain internal system names, unreleased dates or commitments that Engineering has not confirmed.

8. Hotfixes

A hotfix is a fix for a production defect that cannot wait for the next train. Hotfixes are created on `hotfix/<ticket>` from the production tag currently running, so that the fix contains only the intended change.

8.1 Hotfix procedure

1 The engineering manager approves the need for a hotfix and links a NorthDesk ticket (or incident ticket under ENG-004).
2 The developer creates `hotfix/<ticket>` from the production tag and implements the smallest possible fix with a regression test.
3 A reviewer approves the PR. For Tier-1 services two reviewers are still required unless the Incident Commander records that only one is available during a SEV1; in that case the second review must be completed within 1 business day after merge.
4 The release manager tags the fix as a PATCH version with a signed tag and the pipeline builds and deploys it.
5 The change is raised as an Emergency change, approved by the Engineering Director on call, outside the Tuesday to Thursday window if needed.
6 The fix is merged or cherry-picked to `main` within 2 business days, and any open release branches receive it as well.
7 The retrospective for the Emergency change is held within 1 business day.

8.2 Hotfix limits

A hotfix must not include refactors, dependency upgrades or unrelated improvements. If the hotfix touches authentication, cryptography or personal data handling, a Security Champion must review it even in an emergency, and the review may happen right after the merge but before the production rollout in the case of SEV1.

9. Repository Hygiene

- Merged feature branches are deleted automatically.
- Commit messages follow the form `<ticket>: <summary>` and should explain why the change is needed.
- Secrets must never appear in source code; the pipeline's secret scan blocks the PR, and any leaked secret must be revoked and rotated immediately, with the Security Desk informed.
- Large binary files must not be committed. Build artefacts are stored in the artefact repository from NorthShip.
- Dependency additions must follow ENG-015, and a new dependency with an unpatched critical vulnerability is not allowed.
- Inactive repositories are archived after 12 months without commits, with owner approval.

10. Exceptions and Edge Cases

- A repository with a single maintainer cannot satisfy the two-reviewer rule. Such a repository is not allowed to host a Tier-1 service; the Director of Platform Engineering must assign a second maintainer first.
- A long-running migration may need a branch longer than 3 days. The engineer must record the reason and expected end date in the ticket, and merge from `main` into the branch at least daily.
- A release train that falls on a company holiday moves to the next working day within the Tuesday to Thursday window, announced by the release manager at least 3 business days ahead.
- If CAB cannot meet on Monday at 15:00 IST, the CAB chair may approve Normal changes by email the same day; changes submitted after Monday 12:00 IST wait for the next week's train unless approved as Emergency.

- Experimental forks of open-source code must comply with ENG-015 licence rules before they are used in a product.

11. Worked Examples

Example 1 (standard flow): On Monday a developer starts `feature/NX-4701-cursor-limit`, finishes in 2 days, and opens a 220-line PR. Two reviewers approve (Tier-1 service), pipeline gates pass, coverage is 86% and the branch is squash merged on Wednesday. The change joins the next Tuesday train. The Normal change request is submitted by Monday 12:00 IST and approved by CAB at 15:00 IST. On Tuesday it rolls out at 10% for 30 minutes, 50% for 30 minutes and then 100%.

Example 2 (missed deadline): A developer submits a Normal change request at Monday 13:30 IST. The deadline of Monday 12:00 IST has passed, so the change waits for the following week's CAB unless the Engineering Director on call approves it as an Emergency change, which requires a genuine urgent customer or security reason.

Example 3 (Friday request): A product manager asks for a Friday afternoon deployment to meet a demo. The release manager declines because no Friday production deployments are allowed except emergency. The change is scheduled for the next Tuesday train instead.

12. Related Documents

- ENG-001 Code Review Standards: reviewer counts, PR size and security-sensitive reviews.
- ENG-003 Software Deployment Runbook: deployment steps, rollback and change types.
- ENG-007 CI/CD Pipeline Reference: pipeline stages and gates.
- ENG-002 API Development Standards: versioning and deprecation of public APIs.
- ENG-015 Open-Source and Third-Party Library Usage Policy: licence and dependency rules.
- CS-006 Release Communication Guide: customer communication of releases.

13. Revision History

Version	Date	Change
3.0	2025-10-02	Adopted trunk-based development as the default model
3.2	2026-01-21	Added feature flag rules and signed tag requirement
3.3	2026-03-12	Updated product version table and freeze examples